Dialogue Framework

Node-based dialogue system with quest tracking, branching replies, floating dialogues and event-driven runtime for Unity.

Author	**Chapi**
Version	1.1.0
Unity	2022.3 LTS or newer (Unity 6 supported)
Dependencies	TextMeshPro, Input System

Documentation for the Dialogue Framework Unity package, including setup instructions, system architecture, floating dialogues and script reference.

TABLE OF CONTENT

1. Introduction

Dialogue Framework is a complete node-based dialogue system for Unity with built-in quest tracking, branching player replies, floating world-space dialogues and an event-driven runtime. It is designed for narrative-driven games where dialogues need to react to the world: quest states, completed objectives, player choices and events from any system in the game.

The system is structured around a single ScriptableObject asset (`GraphData`) that holds all actors, quests and conversations. Each conversation contains a graph of nodes connected by links. At runtime, a `DialogueManager` reads a specific conversation from the asset and presents it to the player.

1.1. Key features

- **Visual editor** — custom GraphView with drag-and-drop nodes, live dropdowns and auto-save.
- **Multiple conversations** — a single asset can hold any number of named conversations. NPCs reference their conversation by name.
- **Branching replies** — each node can have multiple player reply options with their own output ports.
- **Quest system** — five quest states (NotStarted, Active, InProgress, Completed, Failed) with per-objective progress tracking.
- **Node effects** — actions fired when the player reaches a node: start/complete/fail quests, complete specific objectives.
- **Node requirements** — quest status and objective completion requirements gate node accessibility automatically.
- **Event bus** — name-based static event channel for game-wide events. Inspector bindings let events progress quests.
- **Floating dialogues** — world-space billboard dialogues that trigger on collider enter / exit, perfect for ambient barks and signposts.
- **Bundled UI themes** — ready-made Simple, Cyberpunk and Zelda canvas prefabs.

2. Requirements

- Unity 2022.3 LTS or newer (Unity 6 fully supported).
- TextMeshPro package (auto-imported by Unity).
- Input System package.

3. Installation

Import the package from the Asset Store, or copy the `DialogueFramework` folder into your project's

`Assets` directory. Wait for Unity to compile.

To verify the install, open **Window** → **Dialogue Framework**. If the editor window opens with three tabs (Nodes, Actors, Quests), the installation is complete.

Optionally, generate a fully wired sample by running **Tools** → **Dialogue Framework** → **Generate Example Graph**.

4. Quick Start

This step-by-step tutorial walks through creating a complete dialogue with a quest from scratch. Total time: about ten minutes.

4.1. Open the editor

Open **Window** → **Dialogue Framework**. The window opens with an empty graph asset slot and three tabs.

4.2. Create a graph asset

Click **New**. Choose a location and filename. The editor will create a `GraphData` ScriptableObject and load it automatically.

4.3. Define actors and quests

Switch to the **Actors** tab and click **Add Actor**. Give each actor a name. Actors are shared across all conversations.

Switch to the **Quests** tab. Click **Add Quest** and fill in the title and description. Use **Add Objective** to define the objectives the player must complete.

4.4. Create a conversation

Switch to the **Nodes** tab. Click **+ New Conversation** at the top of the canvas. Use **Rename** to give it a descriptive name such as `Aldric_Forge`. This name is what the runtime uses to find the conversation.

A single asset can hold any number of conversations. Use the dropdown to switch between them. Only the nodes of the active conversation are shown in the canvas.

4.5. Build the dialogue flow

Right-click on the canvas and choose **Create node**. Each node exposes the following fields:

- **Name / Dialogue** — the text spoken by the actor.
- **Actor** — the speaker (or *None* for narration).
- **Objective requirements** — objectives that must be completed (or not) for this node to be reachable.
- **Quest requirements** — quest statuses required for this node to be reachable.
- **Player replies** — branching choices, each with its own output port.
- **Node effects** — actions fired on entering the node: QuestStart, QuestComplete, QuestFail, ObjectiveComplete.

Drag from an output port to another node's input port to connect them. Nodes without incoming links are treated as the start of the conversation.

4.6. Set up the scene

Add three GameObjects to your scene:

GameObject	Component	Purpose
Managers/QuestManager	QuestManager	Tracks quest state. Configure objective bindings to auto-complete
UI/DialogueCanvas	DialogueManager	Runs the dialogue UI. Set m_ConversationName to the conversation.
UI/DialogueCanvas/Input	DialogueController	Handles keyboard/gamepad input. Reads from DialogueManager.

Assign the same `GraphData` asset to the QuestManager and DialogueManager.

4.7. Trigger a dialogue

From any script in your scene, call StartDialogue():

```
dialogueManager.StartDialogue();
```

4.8. Fire game events

When something happens in the world, raise an event by name:

```
GameEventBus.Raise("OnIronCollected");
```

The QuestManager listens to these events and updates objectives automatically, based on the bindings configured in its inspector. Events are ignored if the corresponding quest has not been started yet, so the player cannot complete objectives prematurely.

5. Floating Dialogues

In addition to the standard screen-space dialogue UI, the framework supports **floating dialogues**: world-space text bubbles that live on the NPC, automatically face the player, and trigger when the player enters or exits a trigger collider.

5.1. When to use them

Floating dialogues are ideal for ambient or non-interactive content:

- Wandering NPCs speaking short barks when the player approaches.
- Signs and notice boards with contextual information.
- Environmental storytelling: ghosts whispering, machines humming, creatures reacting to proximity.

Floating dialogues skip the typewriter animation, do not show reply buttons and do not require any player input. They start when the player enters the trigger and end when the player leaves.

5.2. Setup on the NPC

On the NPC GameObject:

- Add a **Collider** component set as trigger (e.g. a SphereCollider with the radius you want the dialogue to appear within).
- Add a **DialogueManager** component, assign its GraphData and pick the conversation from the dropdown.

- Tick the **Is Floating Dialogue** checkbox at the top of the DialogueManager. This skips the typewriter, hides reply buttons and does not register the manager as `DialogueManager.Active`.
- Add a **FloatingDialogueController** component. Drag the in-NPC canvas into its **Canvas** field.

5.3. World-space canvas

Floating dialogues use a smaller, world-space version of the regular dialogue canvas. Create one as a child of the NPC:

- Canvas component with Render Mode = World Space.
- Scale the canvas appropriately (typical values are around 0.005 to 0.01).
- Position it above the NPC's head (or wherever fits the design).
- Inside the canvas, keep only the DialoguePanel, ActorPanel, ActorText and DialogueText

GameObjects. Reply panel, next button and progress bar are not used.

Drag the relevant references into the DialogueManager on the NPC: DialoguePanel, ActorPanel, ActorText, DialogueText. The fields related to navigation (Next button, Replies panel, Reply button prefab) can be left empty.

5.4. Behaviour at runtime

When the player enters the trigger:

- The FloatingDialogueController calls `StartDialogue()` on the manager.
- The conversation is resolved and the start node is shown immediately with the full text (no typewriter animation).
- The world-space canvas billboards toward the player smoothly, rotating on the Y axis only so the text always stays upright.

When the player exits the trigger:

- `EndDialogue()` is called and the canvas is hidden.

The FloatingDialogueController exposes a **m_RotationSmoothing** field to tune how snappy the billboard rotation feels: lower values give a slow, dreamy follow, higher values track the player instantly.

Multiple NPCs can have floating dialogues simultaneously without conflict. Floating dialogues never claim the global `DialogueManager.Active` slot, so a standard interactive dialogue can run at the same time as ambient floating ones.

6. Architecture Overview

The system is split into a **data layer** (the GraphData asset), an **editor layer** (the visual editor and ScriptableObject inspectors) and a **runtime layer** (the MonoBehaviours that drive the dialogue at play time).

Layer	Component	Role
Data	GraphData	ScriptableObject holding actors, quests, conversations, nodes and links.
Editor	EditorWindow	Main visual editor with three tabs.
Editor	DialoguesView	GraphView for dialogue nodes, filtered by conversation.
Editor	ActorsView / QuestsView	List views for actors and quests.
Editor	NodeGraphSaveUtility	Persists graph state into the asset.
Runtime	DialogueManager	Plays a specific conversation. One per dialogue UI or per NPC for floating ones.
Runtime	DialogueController	Standard UI input integration.
Runtime	FloatingDialogueController	Trigger-based floating dialogue with billboard.
Runtime	QuestManager	Singleton tracking quest progress.
Runtime	QuestController	Optional UI panel showing the quest journal.
Runtime	GameEventBus	Static name-based event channel.

7. Editor Reference

7.1. Editor window layout

Open the editor with **Window** → **Dialogue Framework**. The window has a top bar with the asset slot, New/Save buttons, and three tabs:

- **Nodes** — the dialogue graph for the currently selected conversation, with a second bar above the canvas that hosts the conversation dropdown and management buttons.
- **Actors** — flat list of actors (characters).
- **Quests** — flat list of quests, each with its objectives.

7.2. Conversation management

Inside the Nodes tab a **Conversation** dropdown and three buttons let you manage conversations:

- **+ New Conversation** — creates and switches to it.
- **Rename** — renames the active conversation. Make sure any DialogueManager pointing to the old name is updated.
- **Delete** — removes the active conversation and all its nodes and links (with confirmation).

7.3. Node fields

Name / Dialogue — Displayed node name and text shown to the player.

Actor — Dropdown of actors from the Actors tab. Select *None* for narration nodes (the actor name panel hides at runtime).

Objective Requirements — (quest, objective, mustBeCompleted) tuples. The node is only reachable if every requirement is satisfied.

Quest Requirements — (quest, status) tuples. The node is only reachable if every quest is in the specified status.

Player Replies — Branching choices. Each reply adds a dedicated output port. Without any reply the node uses a single generic output.

Node Effects — Actions fired when the player enters this node: QuestStart, QuestComplete, QuestFail, ObjectiveComplete.

8. Runtime Reference

All runtime types live in the `DialogueFramework` namespace.

8.1. DialogueManager

MonoBehaviour that plays a specific conversation from a GraphData asset. Each NPC or dialogue trigger in the scene has its own DialogueManager.

Public fields

- `m_GraphData` — the asset containing the conversation data.
- `m_ConversationName` — conversation to play (e.g. *Aldric_Forge*).
- `m_IsFloatingDialogue` — set to true for floating world-space dialogues (skips typewriter, hides reply UI, does not occupy Active).
- `m_CharDelay`, `m_ProgressBarSmoothing` — typewriter and progress bar tuning.

Public methods

```
public   void   StartDialogue();
public    void    EndDialogue();
public void OnNextPressed();

public   void   OnChangedReplySelection(int   direction);
public void OnReplyPressedAction();

public bool HasVisibleReplies(NodeData node);
```

Static

- `DialogueManager.Active` — the currently active *standard* dialogue (floating dialogues do not claim this slot).

8.2. DialogueController

Translates Input System actions into DialogueManager calls. Reads from `DialogueManager.Active`, so any interactive dialogue in the scene is targeted automatically. Input actions used: `SkipContinue`, `NextReply`, `PreviousReply`.

8.3. FloatingDialogueController

Companion component for floating dialogues. Add it to the NPC alongside a DialogueManager set to *floating*. Wires the trigger collider to dialogue start/end and rotates the world-space canvas to face the player smoothly.

Public fields

- `m_Canvas` — the world-space canvas to rotate (typically a child of the NPC).
- `m_RotationSmoothing` — rotation smoothness (higher = snappier, lower = lazier).

Behaviour: on `OnTriggerEnter` with the Player tag, calls `StartDialogue()`. On `OnTriggerExit`, calls `EndDialogue()`. While the dialogue is running, the canvas smoothly rotates so its forward axis points away from the camera (keeping the text readable).

8.4. QuestManager

Singleton tracking the runtime state of every quest defined in the GraphData. Exposes events for UI integration and reacts to named events via the bindings configured in the inspector.

Public methods

```
void   StartQuest(string   questGuid);
void CompleteQuest(string questGuid);
void FailQuest(string questGuid);

void    SetObjectiveCompleted(string    questGuid,
string objectiveGuid,

bool completed = true);



QuestStatus GetStatus(string questGuid);

bool  IsObjectiveCompleted(string  q,  string  o);
(int,    int)    GetProgress(string    questGuid);
IEnumerable&lt;...&gt; GetAllQuests();
```

Events

```
event Action&lt;string, QuestStatus&gt; OnQuestStatusChanged;
event Action&lt;string, string, bool&gt; OnObjectiveChanged;
```

8.5. QuestController

Optional MonoBehaviour driving a quest journal panel. Lists quests filtered by status, shows their objectives with toggles, and updates live as the QuestManager fires events.

Public methods

```
void     OpenQuestPanel();
void CloseQuestPanel();
```

8.6. GameEventBus

Static class that routes named events between any number of publishers and subscribers, without requiring direct references.

```
static void Raise(string eventName);

static void Subscribe(string eventName, Action callback);
static void Unsubscribe(string eventName, Action callback);
static void Clear();
```

9. Bundled UI Themes

Three ready-to-drop canvas prefab sets are included under `Prefabs/Examples`:

- **Simple** — minimalist clean theme.
- **Cyberpunk** — neon-styled HUD theme.
- **Zelda** — adventure RPG theme inspired by classic console games.

Each theme provides a canvas prefab with dialogue panel and quest journal, plus matching `QuestEntry`, `Objective` and `ReplyButton` prefabs. Drop any of them into the scene and wire it to your DialogueManagers.

10. Extending the System

Add a new effect type

Add a value to the `NodeEffectType` enum in GraphData.cs, then add a matching case in `DialogueManager.ExecuteNodeEffects()`. The editor dropdown regenerates automatically.

Add a new requirement type

Add a new list to `NodeData` following the pattern of `s_QuestRequirements` or `s_ObjectiveRequirements`, and add the matching evaluation to `DialogueManager.NodeConditionsMet()`. Mirror the editor foldout in DialogueEditorNode.cs.

Custom floating dialogue triggers

FloatingDialogueController is intentionally minimal. To start floating dialogues from line of sight, raycasts, distance checks or any other trigger, write a small MonoBehaviour that calls `StartDialogue()` and `EndDialogue()` on the floating DialogueManager.

Replace the UI

DialogueManager, QuestController and FloatingDialogueController are all MonoBehaviours with explicit references. Replace the bundled prefabs, restyle them or rewrite the UI entirely without touching the data layer.

11. Troubleshooting

- ***Q. Dialogue starts but no node is shown.***
  - *The conversation name in the DialogueManager does not match any ConversationData in the asset. Check spelling and case. Conversation names are matched exactly.*
- ***Q. Floating dialogue triggers but no text appears.***
  - Make sure the world-space canvas has been wired into FloatingDialogueController.m_Canvas, that the canvas Render Mode is World Space, and that ActorPanel / ActorText / DialogueText are assigned in the DialogueManager fields.
- ***Q. Floating dialogue text appears mirrored or unreadable.***
  - The billboard rotates the canvas to face away from the camera so its forward axis points outward. If your canvas was authored with text on its back face, flip the canvas's Z scale to -1, or rotate the root 180 degrees on Y.
- ***Q. All objectives in the journal show as not completed even though the quest is Completed.***
  - Use the bundled CompleteQuest method which auto-marks all objectives, or progress them individually via SetObjectiveCompleted. The runtime tracks completion per objective, not at the quest level.
- ***Q. Events have no effect on quests.***
  - Check that QuestManager.m_ObjectiveBindings has an entry for the event name, that the GraphData is assigned, and that the bound quest is at least in Active state (the binding ignores events while the quest is NotStarted).
- ***Q. Reply buttons fire on the first Enter press after skipping text.***
  - Make sure your reply prefab does not use Button.Select() in any of its own logic. The framework highlights replies manually to avoid the EventSystem double-firing the submit action.

12. Support

For bug reports, feature requests or questions, please contact the author **Chapi** via the Asset Store publisher page.

Thank you for using Dialogue Framework.